PYTHON ESTRUTURAS...

If / ELSE

São estruturas de **decisão** usados para executar blocos de código com base em condições (ex: *se* a senha estiver correta, faça o login; *senão*, exiba um erro).

If (Se): Avalia uma condição. Se ela for verdadeira, o bloco de código dentro dele é executado. Se for falsa, o programa simplesmente ignora esse trecho.

Else (Senão): Vem logo após um *if*. Ele define um bloco de código alternativo que será executado apenas se a condição do *if* não for atendida (for falsa).

Operadores de Comparação no Python

Os operadores de comparação são usados para comparar valores e retornam um resultado booleano: `True` (verdadeiro) ou `False` (falso).

Operador	Descrição	Exemplo	Resultado
`==`	Igual a	`5 == 5`	`True`
`!=`	Diferente de	`5 != 3`	`True`
`<`	Menor que	`3 < 5`	`True`
`>`	Maior que	`5 > 3`	`True`
`<=`	Menor ou igual a	`3 <= 3`	`True`
`>=`	Maior ou igual a	`5 >= 3`	`True`

```python
x = 10
y = 5

# Comparações
print(x == y)  # False
print(x != y)  # True
print(x > y)   # True
print(x <= y)  # False
```

Como fazer comparações?

- Os operadores de comparação são usados para comparar valores e retornam um resultado booleano.
- Ou seja, há dois resultados possíveis de uma comparação: True ou False
- Comparações são usadas em estruturas de decisão e de repetição.

Como funciona:

1. O Python verifica a condição no `if`.
2. Se a condição for `True`, o bloco dentro do `if` será executado.
3. Caso contrário, o bloco dentro do `else` será executado.

```python
condicao = True
if (condicao == True):
    # Tudo o que acontece se a
    # condição for verdadeira
    print('Condição Verdadeira')
else:
    # Tudo o que acontece se a
    # condição for falsa
    print('Condição Falsa')
```

O que é uma Estrutura de Decisão?

- Permite que o programa tome decisões com base em condições.
- É como dizer: 'se isso for verdade, faça isso, senão, faça aquilo'.
- Muito usada para controle de fluxo em programas.

```python
nota = 85

if nota >= 90:
    print("Excelente!")
elif nota >= 70:
    print("Bom!")
else:
    print("Precisa melhorar.")
```

Como funciona:

1. O Python verifica a primeira condição no `if`. Se for `True`, executa o bloco associado e ignora os outros.
2. Se o `if` for `False`, o Python verifica as condições no(s) `elif`. O primeiro `elif` verdadeiro é executado.
3. Se nenhuma condição for verdadeira, o bloco `else` é executado.

Elif:

Como funciona?

É usada para verificar múltiplas condições em sequência, permitindo que diferentes blocos de código sejam executados dependendo de qual condição é verdadeira.

O if testa uma condição. O else é o plano B se a condição for falsa. Você também pode usar o elif (abreviação de *else if*) para testar várias condições em sequência.

```
idade = 18

if idade >= 18:
    print("Você é maior de idade. Acesso liberado.")

elif idade >= 16:
    print("Acesso liberado apenas com autorização.")

else:
    print("Você é menor de idade. Acesso negado.")
```

WHILE / FOR

São estruturas de repetição (loops) usados para executar a mesma ação repetidamente.

While (Enquanto): Repete um bloco de código continuamente enquanto uma condição específica continuar sendo verdadeira. É muito útil quando você não sabe exatamente quantas vezes a repetição vai acontecer, mas sabe quando ela deve parar.

For (Para): Executa um bloco de código um número fixo de vezes. Você geralmente já sabe antecipadamente o limite da repetição ou usa ele para percorrer uma lista de itens (como uma lista de e-mails, por exemplo).

O while repete o código enquanto a condição for verdadeira. É ideal quando você não sabe quantas vezes vai repetir.
Nota: É preciso atualizar a variável da condição dentro do bloco para evitar um loop infinito.

```
tentativas = 3

while tentativas > 0:
    print(f"Você tem {tentativas} tentativas.")
    tentativas = tentativas - 1  # Diminui o contador

print("Fim do jogo!")
```

For (Repetição por Contagem ou Lista)
O for em Python é perfeito para percorrer elementos de uma lista ou para repetir ações um número exato de vezes usando a função range().

Exemplo com range() (repetir X vezes):
Repete 3 vezes (de 0 até 2)

```
for i in range(3):
    print(f"Número da linha: {i}")
```

Exemplo com Lista:

```
frutas = ["Maçã", "Banana", "Morango"]
for fruta in frutas:
    print(f"Eu gosto de: {fruta}")
```

A função range() em Python gera uma **sequência de números inteiros** de forma automática e aceita até **três argumentos**: range(início, fim, passo).
Ela é extremamente eficiente porque não cria a lista na memória de uma vez, mas gera os números sob demanda. O argumento de parada (fim) é **exclusivo**, o que significa que o Python para um número antes do valor definido.

Um argumento: range(fim)

Gera números começando em **0** e vai até o valor definido (menos um).

```
# Gera: 0, 1, 2, 3, 4
for i in range(5):
    print(i)
```

Dois argumentos: range(início, fim)

Permite escolher onde a contagem **começa**, em vez de iniciar obrigatoriamente no zero.

```
# Começa no 2 e para no 6 (Gera: 2, 3, 4, 5, 6)
for i in range(2, 7):
    print(i)
```

Três argumentos: range(início, fim, passo)
O terceiro argumento define o intervalo (pulo) entre os números da sequência.

```
# Números ímpares de 1 a 9 (Gera: 1, 3, 5, 7, 9)
for i in range(1, 10, 2):
    print(i)
```

Contagem regressiva (Passo negativo)
Número negativo no terceiro argumento, a range() faz uma contagem decrescente.
O valor de início deve ser maior que o de fim.

```
# Contagem regressiva (Gera: 5, 4, 3, 2, 1)
for i in range(5, 0, -1):
    print(i)
```

MATTHES, E. *Curso Intensivo de Python*: Uma Introdução Prática e Baseada em Projetos à Programação. São Paulo: Novatec, 2016
MENEZES, N. N. C. *Introdução à Programação com Python*: Algoritmos e Lógica de Programação. São Paulo: Novatec, 2019.
CRUZ, F. *Python*: escreva seus primeiros programas. São Paulo: Casa do Código, 2016.
Python Profissional: do zero ao mercado de trabalho! - Dilermando Piva Junior Professor | Autor | Data Scientist.
Python Completo do Zero ao Avançado + Projetos Reais 2025 - Andre Iacono AI · Cloud · Developer |Educating with Passion, Transforming with Purpose. Programação em Python do básico ao avançado - Geek University.

ESTRUTURAS DE DECISÃO E REPETIÇÃO

IF / ELSE – SE / SENÃO

EXECUTA SÓ UMA VEZ... (VERDADEIRO OU FALSO).

FOR / LOOP – PARA / PARA CADA

PARA ESSA VARIÁVEL GIRAR TANTAS VEZES (GIRA POR N VEZES - DETERMINADO).
NÃO SEI O TAMANHO DO LOOP.

WHILE / LOOP - ENQUANTO

PARA ESSA VARIÁVEL, SE X OU Y, GIRE TANTAS VEZES ATÉ ATENDER SITUAÇÃO, (CRITÉRIO) INDETERMINADO ATÉ ATENDER A CONDIÇÃO.